



K8S

Kubernetes

container orchestration tool

What is Kubernetes ?



→ orchestration

- Portable, extensible, open-source platform for managing containerized workloads and services
- Facilitates both declarative configuration and automation
- It has a large, rapidly growing ecosystem
- Kubernetes services, support, and tools are widely available
- The name Kubernetes originates from Greek, meaning helmsman or pilot
- Google open-sourced the Kubernetes project in 2014

✓ declaration → YAML file
automation → CLI
} → kubectl

K8s installation

→ on prem → self managed

→ cloud

→ self managed → cluster using VMs

→ managed service → managed by cloud provider → PaaS → Preferred

→ AWS → EKS

→ Azure → AKS

→ GCP → GKE

k8s cluster

→ single node cluster

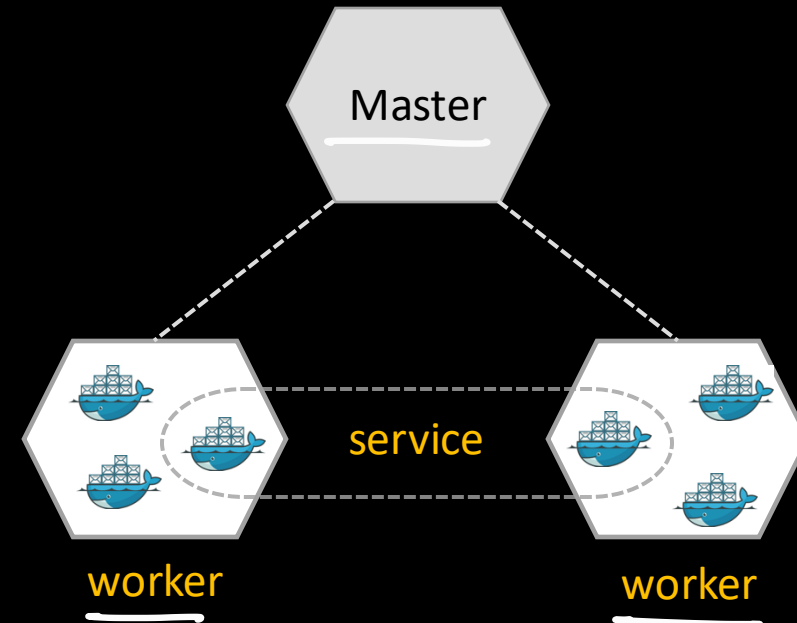
- ↳ minikube → simulated cluster
- ↳ used only for learning purpose

→ multi node cluster (real cluster)

- ↳ single master cluster
 - ↳ used for development or testing
- ↳ multi master cluster (HA cluster)
 - ↳ used for production environment
 - ↳ Election will choose Leader
 - ↳ algo → Raft consensus

Kubernetes Cluster

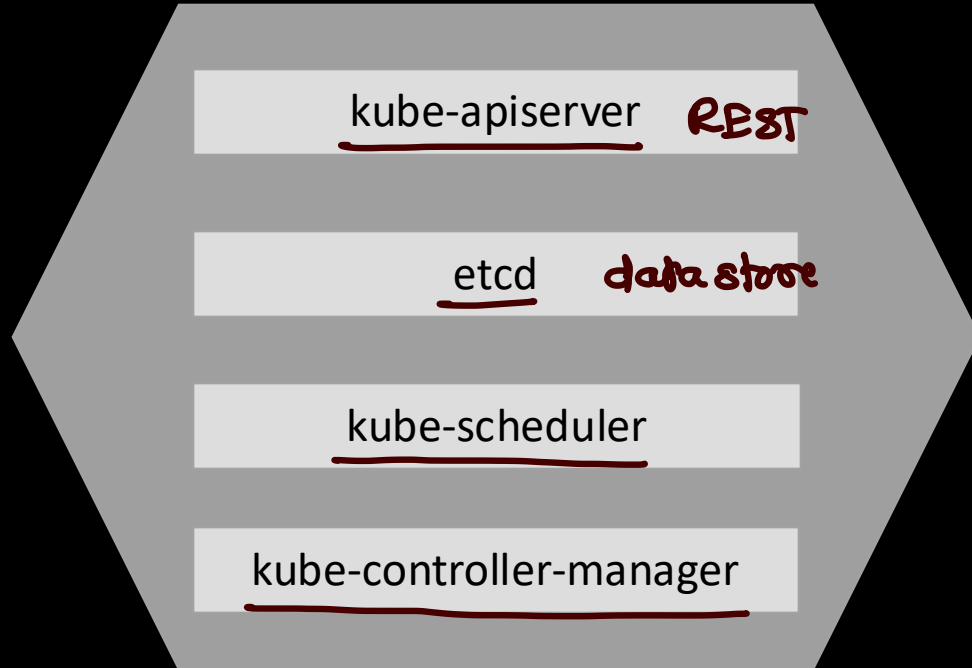
- When you deploy Kubernetes, you get a cluster.
- A cluster is a set of machines (nodes), that run containerized applications managed by Kubernetes
- A cluster has at least one worker node and at least one master node
- The worker node(s) host the pods that are the components of the application
- The master node(s) manages the worker nodes and the pods in the cluster
- Multiple master nodes are used to provide a cluster with failover and high availability



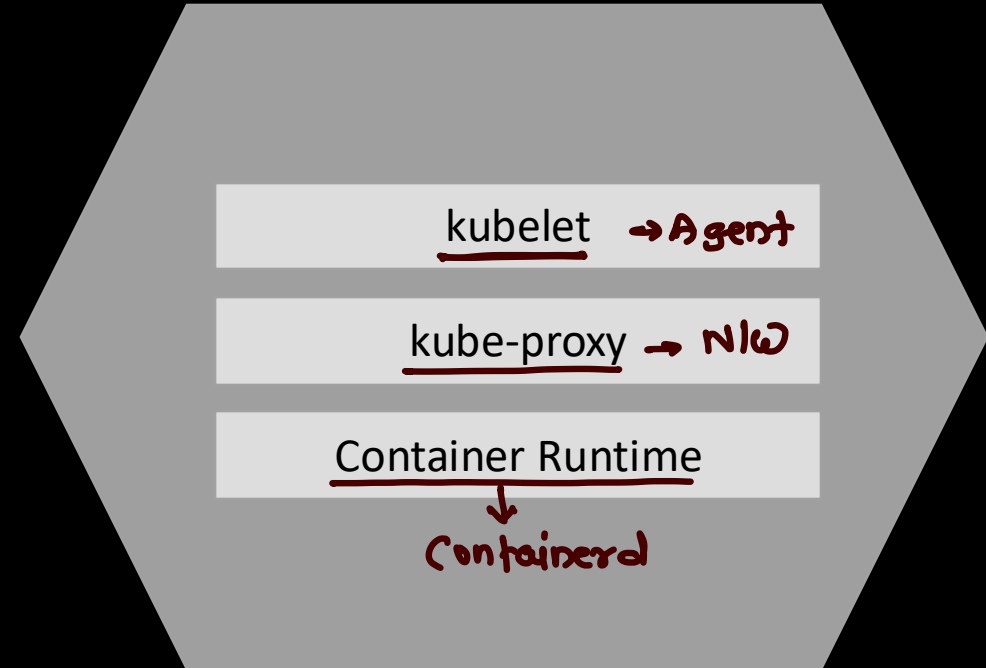
Kubernetes Components

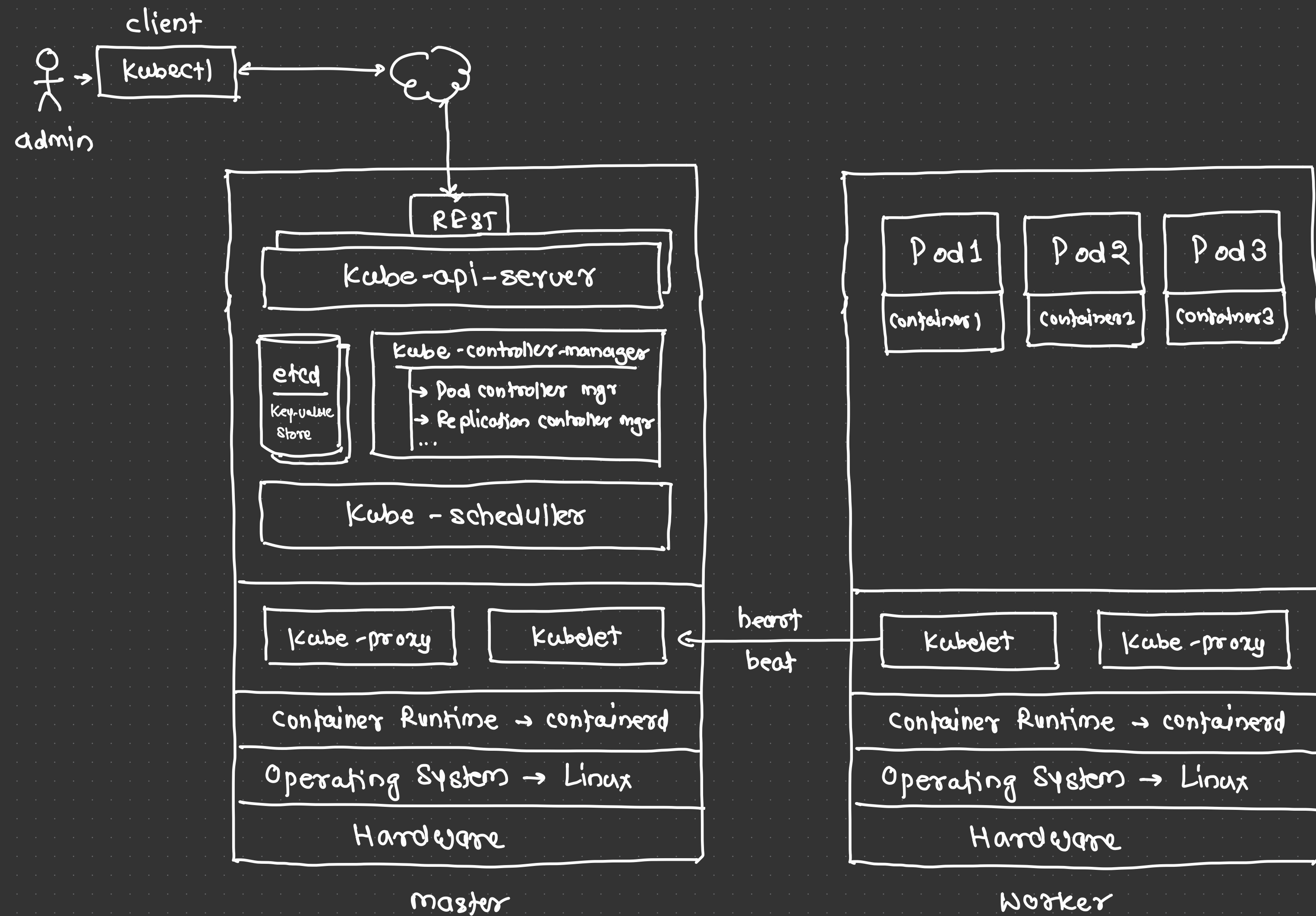


Master



Node





Master Components



- These are the brain of the Kubernetes cluster — they make all the decisions about what runs where, how scaling happens, and how the system stays healthy.
- Master components can be run on any machine in the cluster

Component	Function
<u>kube-apiserver</u>	<u>The front door of the Kubernetes control plane. It exposes the Kubernetes API, which is used by all components, kubectl, and external clients.</u>
<u>etcd</u> → Highly Available Key-value data store	<u>The database of Kubernetes — stores all cluster data, configurations, and states in a key-value format.</u>
<u>kube-scheduler</u>	<u>Decides which node each Pod should run on based on resource requirements and constraints.</u>
<u>kube-controller-manager</u>	<u>Ensures that the actual state of the cluster matches the desired state defined in manifests.</u>
<u>cloud-controller-manager</u> (optional)	<u>Manages integration between Kubernetes and your cloud provider (e.g., AWS, GCP, Azure).</u>

Master Components



■ kube-apiserver

- Acts as the communication hub between users, components, and the cluster.
- Every kubectl command goes through it.
- Validates requests and updates etcd accordingly.
- It's a stateless service — you can run multiple instances for high availability

■ etcd

- A distributed, consistent key-value store.
- Stores all cluster data, including:
 - Pod states
 - Configurations
 - Secrets
 - Node information
- It's the source of truth for your cluster.
- It's critical — if etcd is lost, your cluster loses its state.

Master Components



■ kube-scheduler

- Watches for new Pods that don't have a Node assigned.
- Chooses the best Node to run the Pod based on:
 - Resource requests (CPU, memory)
 - Node affinity/anti-affinity
 - Taints and tolerations
 - Pod priorities
 - Custom policies

■ kube-controller-manager

- Runs a set of controller loops, each responsible for maintaining part of the system's desired state.
- Examples of controllers:
 - Node Controller — manages node status.
 - Replication Controller — ensures the desired number of pod replicas are running.
 - Endpoint Controller — manages endpoint objects.
 - Service Account & Token Controllers — create default accounts and tokens.

↳ used for security → RBAC

Node Components



- Node components run on every node, maintaining running pods and providing the Kubernetes runtime environment
- **kubelet**
 - Primary agent that runs on every node.
 - Communicates with the API Server.
 - Ensures the containers defined in PodSpecs are running and healthy.
 - Reports node and pod status back to the control plane. → *master*
 - Watches for Pod definitions assigned to the node and runs them using the container runtime.
- **kube-proxy**
 - Maintains network rules on nodes.
 - Ensures that networking is consistent across the cluster.
 - Implements Kubernetes Service abstraction — enabling communication between different Pods and Services.
 - Uses iptables or IPVS to forward traffic to the correct Pod endpoints. → *fire wall*
 - Supports load balancing between Pods behind a Service.
- **Container Runtime**
 - The actual software responsible for running containers.
 - The kubelet interacts with the runtime through the Container Runtime Interface (CRI).
 - Common runtimes:
 - containerd (default on most modern clusters)
 - CRI-O
 - Docker Engine (deprecated as of K8s v1.24+)
 - Mirantis Container Runtime

Create Cluster



- Use following commands on both master and worker nodes

```
> sudo apt-get update && sudo apt-get install -y apt-transport-https curl
```

```
> curl -s https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key add -
```

```
> cat <<EOF | sudo tee /etc/apt/sources.list.d/kubernetes.list deb https://apt.kubernetes.io/kubernetes-xenial main EOF
```

```
> sudo apt-get update
```

```
> sudo apt-get install -y kubelet kubeadm kubectl
```

```
> sudo apt-mark hold kubelet kubeadm kubectl
```



Initialize Cluster Master Node

- Execute following commands on master node

```
> kubeadm init --apiserver-advertise-address=<ip-address> --pod-network-cidr=10.244.0.0/16
```

```
> mkdir -p $HOME/.kube
```

```
> sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

```
> sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

- Install pod network add-on

```
> kubectl apply -f
```

```
https://raw.githubusercontent.com/coreos/flannel/2140ac876ef134e0ed5af15c65e414cf26827915/Documentation/kubernetes-flannel.yml
```

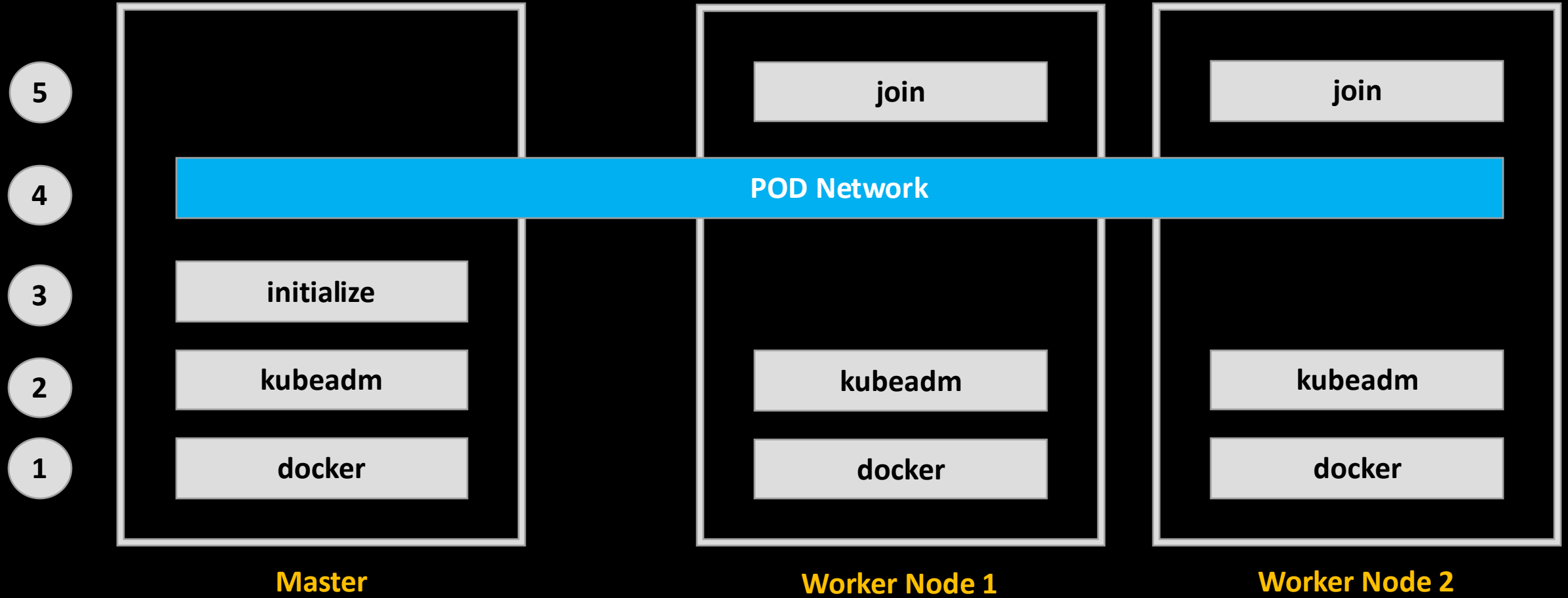
Add worker nodes



- Execute following command on every worker node

```
> kubeadm join --token <token> <control-plane-host>:<control-plane-port> --discovery-token-ca-cert-hash sha256:<hash>
```

Steps to install Kubernetes



Kubernetes Objects



- The basic Kubernetes objects include

- ✓ Pod
- ✓ Service
 - Volume
- ✓ Namespace

- Kubernetes also contains higher-level abstractions build upon the basic objects

- ✓ Deployment
 - DaemonSet
 - StatefulSet
- ✓ ReplicaSet
 - Job

Namespace → group of objects



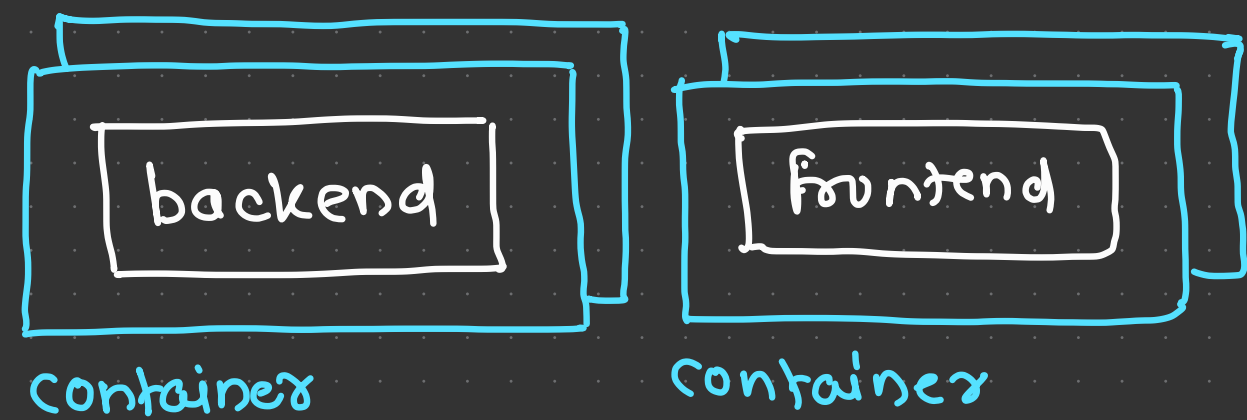
- Namespaces are intended for use in environments with many users spread across multiple teams, or projects
- Namespaces provide a scope for names
- Names of resources need to be unique within a namespace, but not across namespaces
- Namespaces can not be nested inside one another and each Kubernetes resource can only be in one namespace
- Namespaces are a way to divide cluster resources between multiple users (projects) environments

Pod

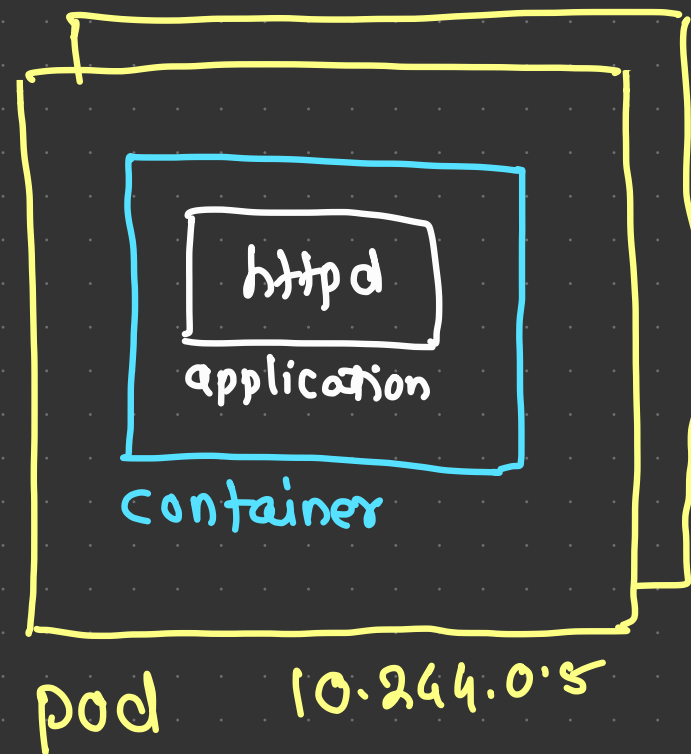


- A Pod is the basic execution unit of a Kubernetes application
- The smallest and simplest unit in the Kubernetes object model that you create or deploy
- A Pod represents processes running on your Cluster
- Pod represents a unit of deployment
- A Pod encapsulates
 - application's container (or, in some cases, multiple containers)
 - storage resources
 - a unique network IP
 - options that govern how the container(s) should run

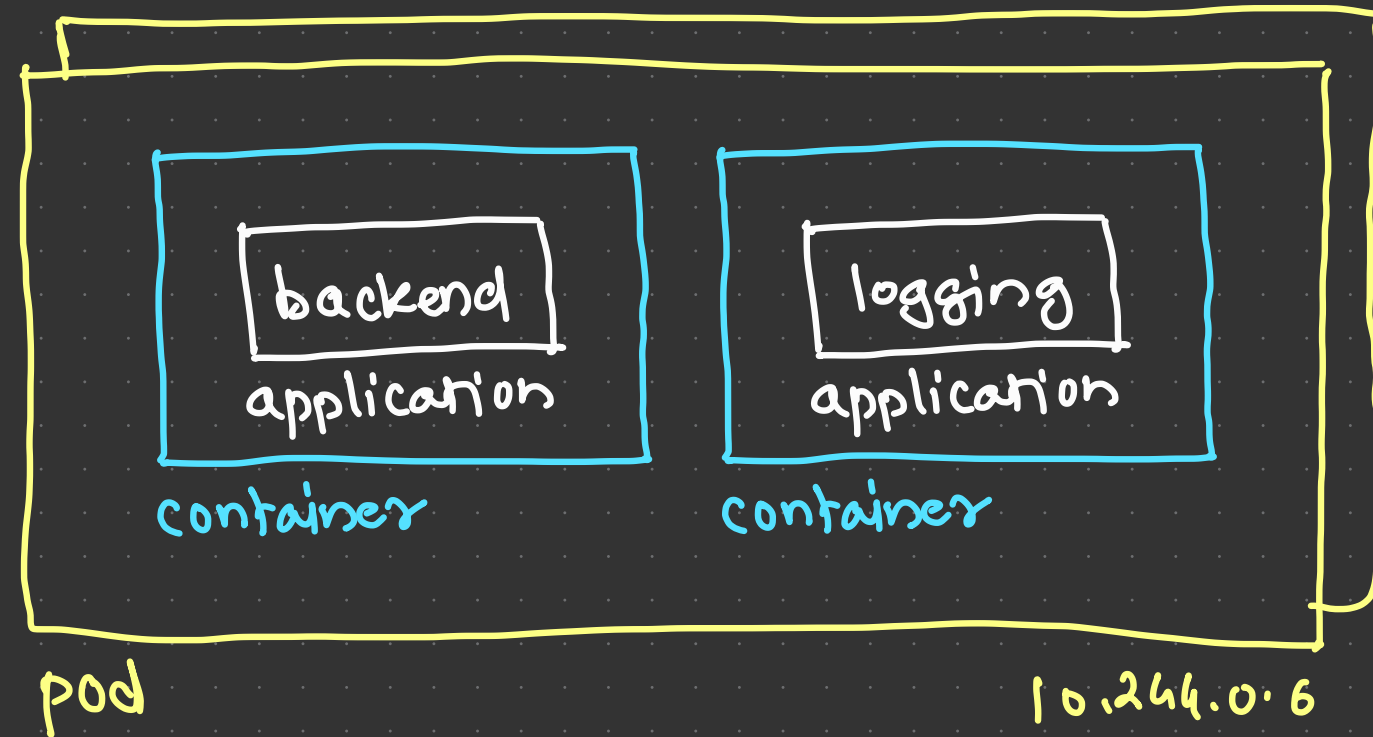
pod types



Single container pod



multi container pod



- side car pattern → multiple containers run simultaneously
- init container → only one (main) container will run after init container finishes its execution

YAML to create Pod

apiVersion: v1

kind: Pod

metadata:

name: myapp-pod

labels:

app: myapp

spec:

containers:

- name: myapp-container

image: httpd

* **apiVersion:** v1 (basic) or apps/v1 (higher level)

* **kind:** kind of object to be created

* **metadata:** meta information about the object
↳ name, labels, namespace

* **spec:** specification of object to be created



Service



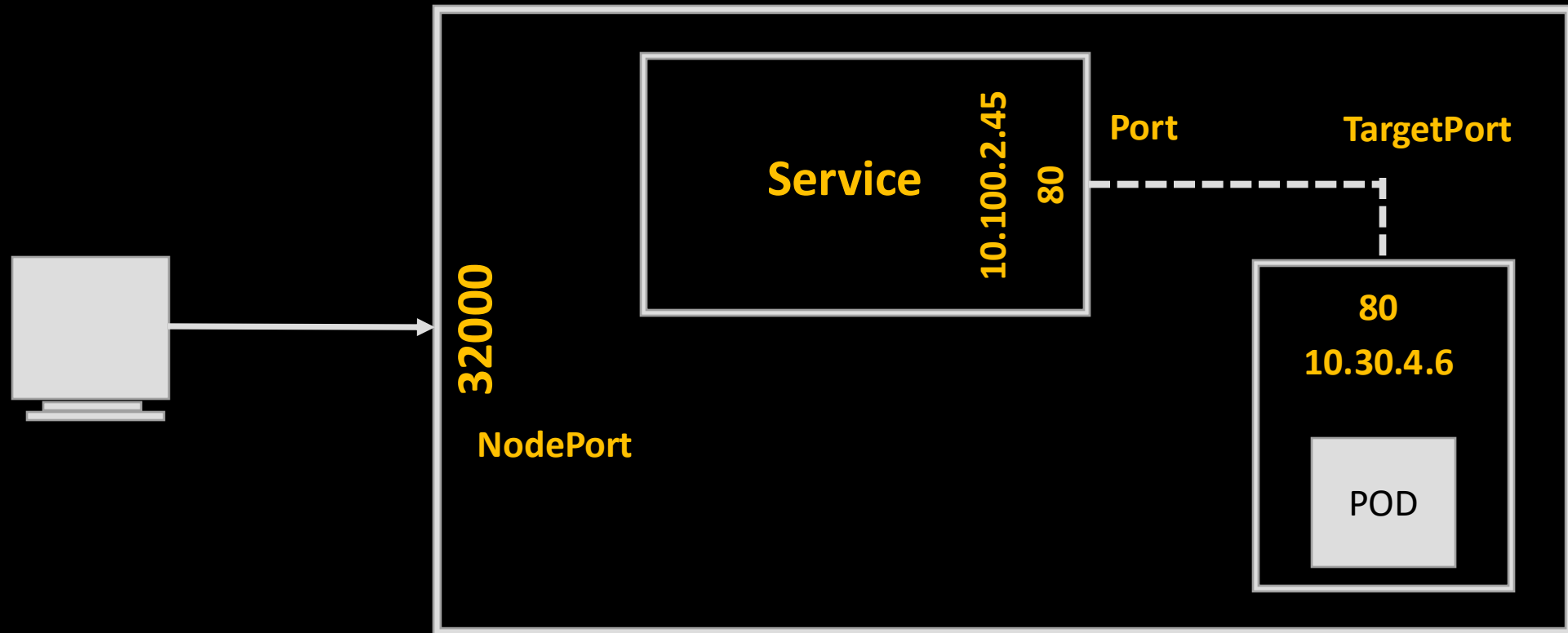
- An abstract way to expose an application running on a set of Pods as a network service
- Service is an abstraction which defines a logical set of Pods and a policy by which to access them (sometimes this pattern is called a micro-service)
- Service Types
 - ClusterIP
 - Exposes the Service on a cluster-internal IP
 - Choosing this value makes the Service only reachable from within the cluster
 - LoadBalancer
 - Used for load balancing the containers
 - NodePort

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
```

Service Type: NodePort



- Exposes the Service on each Node's IP at a static port (the NodePort)
- You'll be able to contact the NodePort Service, from outside the cluster, by requesting <NodeIP>:<NodePort>

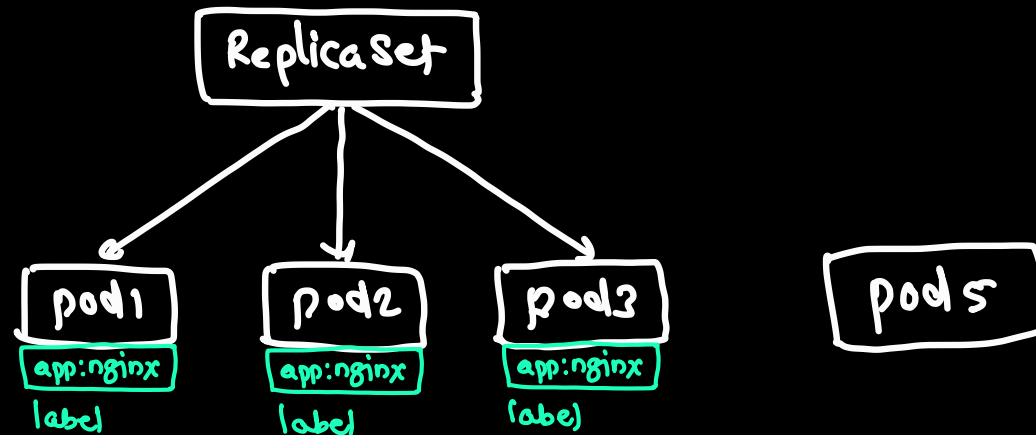


Replica Set



→ desired count

- A Replica Set ensures that a specified number of pod replicas are running at any one time
- In other words, a Replica Set makes sure that a pod or a homogeneous set of pods is always up and available
- If there are too many pods, the Replica Set terminates the extra pods
- If there are too few, the Replica Set starts more pods
- Unlike manually created pods, the pods maintained by a Replica Set are automatically replaced if they fail, are deleted, or are terminated



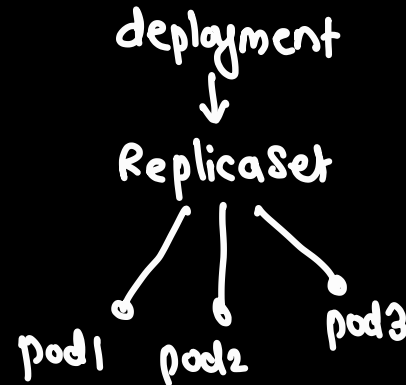
pod definition

```
apiVersion: v1 apps/v1
kind: ReplicaSet
metadata:
  name: nginx
spec:
  replicas: 3 ***
  selector:
    matchLabels:
      app: nginx
  template: to create new pod(s)
    metadata:
      name: nginx
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
```

Deployment



- A Deployment provides declarative updates for Pods and ReplicaSets
- You describe a *desired state* in a Deployment, and the Deployment Controller changes the actual state to the desired state at a controlled rate
- You can use deployment for
 - Rolling out ReplicaSet
 - Declaring new state of Pods
 - Rolling back to earlier deployment version
 - Scaling up deployment policies
 - Cleaning up existing ReplicaSet



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: website-deployment
spec:
  selector:
    matchLabels:
      app: website
  replicas: 10
  template:
    metadata:
      name: website-pod
    labels:
      app: website
    spec:
      containers:
        - name: website-container
          image: pythoncpp/test_website
          ports:
            - containerPort: 80
```